CASE STUDY · SYDNEY, AUSTRALIA

● 04:02:17 AEDT LIVE

AeroSurf

BlueGuard

One season to make BlueGuard trustworthy: every inspection captured, every change kept, every mistake recoverable. Then at 4 AM, one command marked every held asset available, and put damaged gear back on the racks.

ASSETS ON HOLD · 04:00

0

HOLDS CLEARED · UPDATE · EVERY ASSET AVAILABLE

RACK SCAN · MAROUBRA



RACK STATUS · 06:00 BRIEF

BONDI	MANLY	COOGEE	MAROUBRA	CRONULLA
HELD?	HELD?	HELD?	HELD?	HELD?

THE BUILD, IN FOUR PARTS

I Land the drops II Track the change

III Survive the mistake IV Prove it to the regulator

PLATFORM TELEMETRY

TABLE asset_status

LAT/LON 33.89°S 151.27°E

CHANGE FEED on

AUDIT MON 09:00 AEDT

PATROL MAP · SYDNEY AU 33.89°S 151.27°E



SETTING & BACKGROUND · SYDNEY

One season to earn the audit.

Mara Whitlock is Lead Data Engineer at **AeroSurf NSW**. Her platform, **Project BlueGuard**, decides which hire gear is safe to put in the water: boards, wetsuits and jetskis, inspected each shift across Bondi, Manly, Coogee, Maroubra and Cronulla. Anything **damaged** goes on **hold**, never out.

This season she is doing two hard things at once: building a pipeline that takes inspection files as they land and keeps every asset's history, and getting it past a **safety audit**.

So she builds for it: ingestion that grows as the inspection app changes, a history the auditor can read, and recovery designed in before it is ever needed.

Midway through the season, at 4 AM, all of it is tested at once. One command marks every asset available, and three **damaged assets** say they are ready to hire. What Mara built in calm is about to earn its audit in a crisis.

46
INSPECTIONS LOGGED ACROSS THE RUNS

3
DAMAGED ASSETS THE UPDATE FREED

5
BEACHES, ONE SHARED TABLE

WHAT MARA BUILDS, IN FOUR PARTS

I · Land the drops. Take inspection files as they arrive, and let the schema grow.

II · Track the change. Version each asset's condition, keep the full history.

III · Survive the mistake. See the 4 AM write and undo it, no backup.

IV · Prove it to the regulator. Freeze the corrupted state as immutable evidence.

THE DATA

Inspection JSON batches land in `raw_inspections` through Auto Loader, five files a shift, one per beach. The Silver `asset_status` tracks each asset's condition and hire status over time, with the change feed on from creation.

PART I · LAND THE DROPS

Files that keep arriving.

Inspectors drop one JSON file per beach each shift into a landing folder. Newer inspection apps add a field, `defect_notes`. A strict read of a fixed schema would reject those files, and a full re-read every run would reprocess the whole season.

Auto Loader watches the folder. A checkpoint remembers which files it has read, so each run takes only new arrivals. A schema location plus `mergeSchema` let the table gain the new column when it first appears, and older rows read null.

EXHIBIT · TWO SCHEMAS

```
base · runs 1 and 2
{ asset_id, beach, inspected_ts,
  category, condition, hire_status }

run 3 · adds one field
{ ...same six,
  defect_notes }
```

THE MECHANISM

A `cloudFiles` stream reads the folder with a `checkpointLocation` and a `schemaLocation`. `trigger(availableNow=True)` processes the new files this run, then stops, so the same job is incremental run after run. On the write, `mergeSchema` lets `defect_notes` land with no rewrite.

DECISION

Auto-evolve the schema, or gate it?

Auto-evolving means the inspection app can change without a 4 AM page, but a typo'd field name silently becomes a new column and the brief quietly loses data. Gating each change catches every drift, at the cost of a failed run whenever the app legitimately adds a field. Where does Mara draw the line for a safety feed?

Miss this and the run either fails when a new field lands, or silently drops it. Ingestion is a safety decision, not a convenience.

PART II · TRACK THE CHANGE

When did A007 break?

The audit turns on a question: not just that a jetski is damaged now, but **when** it broke and when it went on hold. So `asset_status` cannot hold only the latest state; it must keep history.

This is **SCD Type 2**. When condition or hire status changes, the current row is closed and a new one opened, each version stamped with a surrogate key. One `MERGE` does it, with a catch: it cannot expire and insert the same key at once, so each changed asset is fed to it **twice**.

EXHIBIT · A007, ACROSS RUNS

CONDITION	FROM	TO	
good	run 1	run 2	old
damaged	run 2	null	now

A007 · Coogee board · now on hold

THE MECHANISM · THE TWO-ROW TRICK

```
-- source = each changed asset twice: mergeKey = asset_id, and mergeKey = NULL
MERGE INTO asset_status t USING two_row_src s ON t.asset_id = s.mergeKey AND t.is_current
WHEN MATCHED AND (t.condition<>s.condition OR t.hire_status<>s.hire_status)
  THEN UPDATE SET is_current=false, end_ts=s.inspected_ts -- expire old
WHEN NOT MATCHED THEN INSERT (...) -- insert new version (the NULL-key row)
```

DECISION

The two-row MERGE, or two plain writes?

The single MERGE with the null-key union does the close and the open in one atomic step, but it is the hardest thing in the notebook to read. Splitting it into a close statement and an insert is far clearer, at the cost of two writes and a moment where the change is half applied. Which does Mara hand the next engineer to maintain?

Overwrite instead of version, and the audit can never answer when a board became unsafe. The history is the point.

PART III · SURVIVE THE MISTAKE

The command with no WHERE.

At 4 AM, clearing a backlog, the operator runs `UPDATE asset_status SET hire_status = 'available'` and forgets the filter. Every held asset is now available, including the three that are damaged. There is no typo to catch, because `available` is a valid status.

The **change feed**, enabled when the table was created, captured the write row by row. Read the preimage and postimage of that version and the damage is exact. Then `RESTORE` the table to the version just before the incident. The corrupted version stays in history.

EXHIBIT · THE ROW-LEVEL DAMAGE

ASSET	WAS	NOW
A007	hold	available
A010	hold	available
A011	hold	available

change feed · preimage vs postimage

THE MECHANISM

`table_changes('asset_status', v, v)` returns the `update_preimage` and `update_postimage` for the incident version, the before and after of every touched row. `RESTORE TABLE asset_status TO VERSION AS OF v-1` puts the table back to the state just before the command. RESTORE is itself a new commit.

DECISION

RESTORE the whole table, or repair the three rows?

A full RESTORE is one line and returns the table to a known-good version, but it also unwinds any legitimate write that landed after the incident. Repairing only the three rows from their preimage is surgical and keeps later work, at the cost of trusting you found every affected row. Which is safer when the clock reads 5:50 and the brief ships at six?

The change feed had to be on before the mistake, not after. Recovery is designed in, not bolted on.

PART IV · PROVE IT TO THE REGULATOR

Freeze the evidence.

The auditor wants proof of exactly what the table said during the breach, standing apart from the live table that has since been recovered. A description of the incident is not evidence. An immutable copy is.

A **deep clone** of the corrupted version copies its data and metadata into an independent table that no later change can touch. A **shallow clone**, which shares the source files, is a cheap sandbox for experiments, not evidence, because the files it points at can move or be vacuumed.

EXHIBIT · DEEP VS SHALLOW

	DEEP	SHALLOW
data	copied	shared files
independent	yes	no
use	evidence	sandbox

frozen at the corrupted version

THE MECHANISM

`CREATE TABLE incident_snapshot DEEP CLONE asset_status VERSION AS OF v` freezes the corrupted state as an independent table, the evidence the audit holds. `SHALLOW CLONE` spins a throwaway copy that shares files, useful to test a fix without touching the live table.

DECISION

Deep clone the incident, or trust time travel?

Time travel already lets you read the corrupted version, so a clone can feel redundant. But time travel is only as durable as the retention window, and a VACUUM can remove the files the old version depends on. A deep clone is a standalone table that survives retention, at the cost of storage. What does a regulator accept as evidence months later?

The live table moved on. The deep clone is the frozen truth the audit can hold, long after the window closes.

YOU ARE MARA · PREPARATION QUESTIONS

Build it to survive the season.

Bring the commands and the evidence, not prose. The audit is Monday; the corruption is tonight.

I · LAND. Ingest the inspection folder with Auto Loader, incrementally across the runs. Auto-evolve or gate the new field, and where do you draw the line?

II · TRACK. Build the SCD2 history with a surrogate key. Write the MERGE, and defend which changes earn a new version.

III · SURVIVE. Reverse the 4 AM UPDATE. RESTORE or surgical repair? Give both commands and the risk of each, and name what had to be enabled to see it.

IV · PROVE. Freeze the corrupted version for the auditor. Deep or shallow clone, and why is the other one not evidence?

V · THE GATE. A CHECK constraint on hire_status would not have stopped this. Explain why, and name the control that would have.

VI · THE BRIEF. It is 5:50. What ships for the six o'clock brief, and what do you tell the captains about confidence?

RESOLUTION · SYDNEY, AUSTRALIA

• 06:00:00 AEDT VERIFIED

By six, the racks told the truth.

DAMAGED ASSETS · HELD

3

RESTORED · VERIFIED · EVIDENCE FROZEN

RACK SCAN · MAROUBRA



RACK STATUS · 06:00 BRIEF

BONDI	MANLY	COOGEE	MAROUBRA	CRONULLA
OPEN	OPEN	OPEN	CLOSED	OPEN

Because every write to a Delta table is a versioned commit, nothing Mara built in calm failed her in the crisis. She landed, tracked, recovered and proved, and the platform, not a memo, was the evidence.

BUILD LOG

LAND Auto Loader
TRACK SCD2
SURVIVE RESTORE
PROVE DEEP CLONE
FEED CDF on

I Land the drops.

II Track the change.

III Survive the mistake.

IV Prove it to the regulator.